IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

The Yenepoya institute of arts , science, commerce and management

Project – High Level Design

on

AI-Enhanced CI/CD Pipeline for EdTech Software

Student Name: Keerthi

Industry Mentor: Mr.Shashank **Guided by:Ms.Manjula**

Table of Contents

1. Introduction. 3
   1.1. Scope of the document. 3
   1.2. Intended Audience.... 3
   1.3. System overview.... 3

2. System Design. 4
   2.1. Application Design....4
   2.2. Process Flow.... 5
   2.3. Information Flow.... 10
   2.4. Components Design 10
   2.5. Key Design Considerations 11
   2.6. API Catalogue. 11

3. Data Design.... 12
   3.1. Data Model....12
   3.2. Data Access Mechanism.... 12
   3.3. Data Retention Policies 12
   3.4. Data Migration 12

4. Interfaces....13

5. State and Session Management....13

6. Caching 13

7. Non-Functional Requirements 13
   7.1. Security Aspects....13
   7.2. Performance Aspects.... 13

8. References....14

1. Introduction

This document presents the High-Level Design for an AI-Enhanced Continuous Integration and
Continuous Deployment (CI/CD) Pipeline specifically designed for Educational Technology (EdTech) software applications. The system automates the software delivery lifecycle by integrating build automation, testing frameworks, AI-powered code analysis, and deployment orchestration into a unified pipeline.

The pipeline implements a four-stage architecture comprising Build Stage, Test Stage, AI Analysis Stage, and Deploy Stage. Each stage performs specific quality checks and gates to ensure that only validated, high-quality code reaches production environments. The AI Analysis component employs rule-based static code analysis to identify code quality issues, enforce coding standards, and suggest improvements before deployment.

1.1 Scope of the Document

This document covers the architectural design, system components, process flows, data design, and integration points for the CI/CD pipeline. It includes detailed specifications for each pipeline stage, the AI analysis engine, deployment mechanisms, and quality gates. The document also addresses non-functional requirements including security, performance, scalability, and reliability considerations. It serves as a blueprint for implementation teams and provides technical reference for system maintainers.

1.2 Intended Audience

This document is intended for software architects, development team leads, DevOps engineers, quality assurance teams, project managers, and technical stakeholders involved in the implementation, deployment, and maintenance of the CI/CD pipeline system. It assumes familiarity with software development practices, version control systems, automated testing frameworks, and deployment strategies.

1.3 System Overview

The AI-Enhanced CI/CD Pipeline is a comprehensive automation framework designed to streamline the software development lifecycle for EdTech applications. The system operates as an event-driven pipeline triggered by code commits to version control repositories. It executes a sequential workflow of stages, each implementing specific

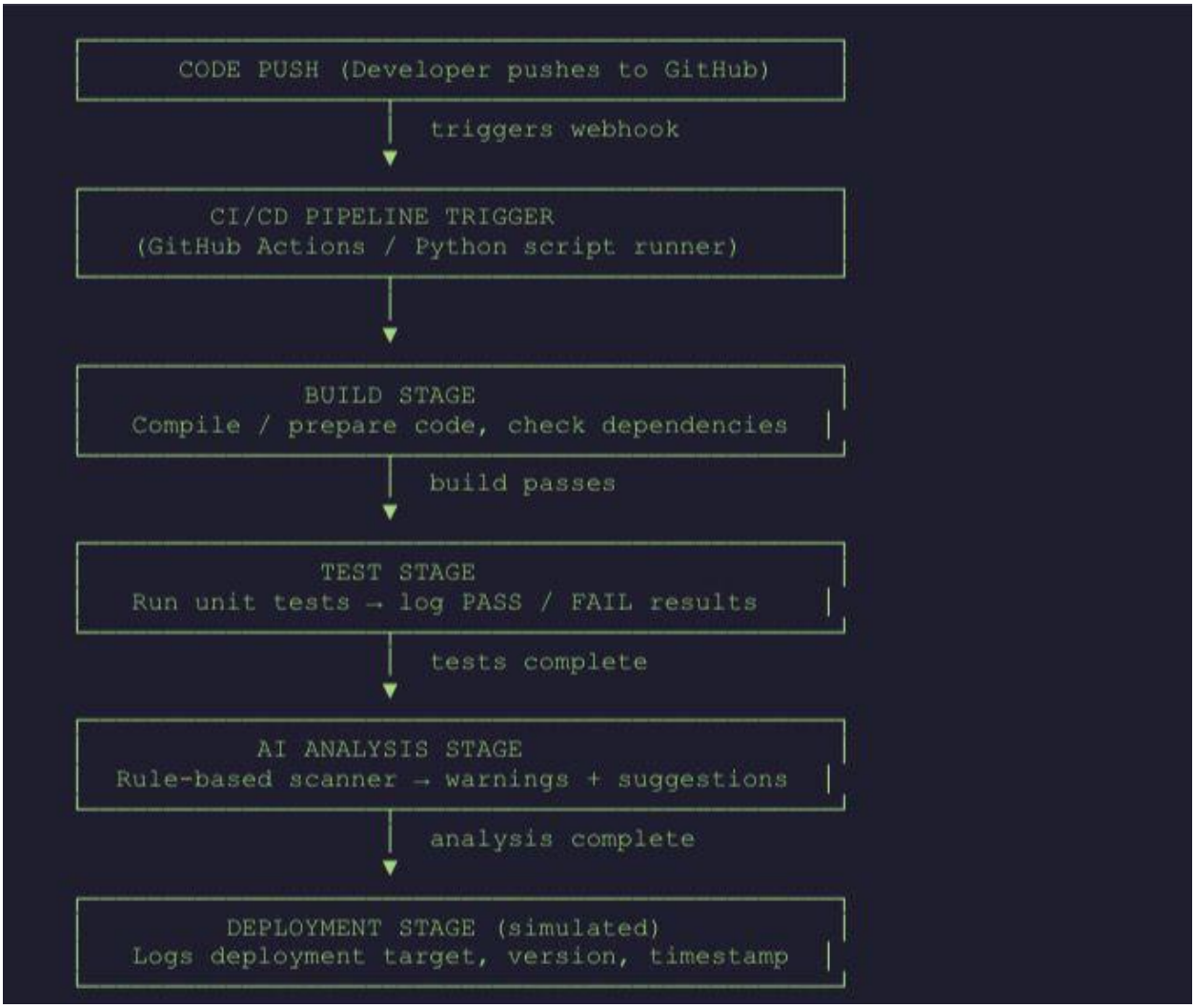
quality gates and automated checks. The pipeline integrates with GitHub Actions for cloud-based CI/CD orchestration while maintaining compatibility with local execution environments for development and testing purposes. The system architecture emphasizes modularity, enabling independent development and testing of each pipeline stage while maintaining cohesive end-to-end workflow execution.

2.System Design

2.1 Application Design

The application follows a modular pipeline architecture with clearly separated concerns. The main orchestrator (run_pipeline.py) coordinates the execution of four independent stage modules. Each stage module exposes a standardized interface with a main execution function that returns a boolean success indicator and optional status messages. The EdTech application under test (sample_code/edtech_app.py) implements core educational platform functionality including student management, course operations, and assessment handling. The pipeline processes this application through sequential quality gates before deployment approval

2.2 Process Flow



Stage 1 - Build Stage: Validates Python runtime environment and verifies presence of required source files. Checks Python version compatibility and ensures all pipeline components are accessible. Fails fast if prerequisites are not met.

```
============================================================
   AI-ENHANCED CI/CD PIPELINE - EdTech Project
   Student : Keerthi
   Course  : BCA Final Year | 2024-25
   Dept    : Computer Science
============================================================
   Started : 2026-04-16 21:07:39
============================================================


------------------------------------------------------------
  >>> STAGE 1: BUILD <<<
------------------------------------------------------------
  Checking Python version...
  [INFO]    Python version: 3.13.7
  [OK]      Python 3.x detected - acceptable!

  Checking required project files...
  [OK]      Found: sample_code/edtech_app.py
  [OK]      Found: tests/test_edtech_app.py
  [OK]      Found: pipeline/ai_analysis_stage.py

  [BUILD PASSED] All checks successful!

  [ PASSED ] BUILD completed successfully!
```

Stage 2 - Test Stage: Discovers and executes all unit tests in the tests directory using Python unittest framework. Runs comprehensive test suite covering student management, course operations, and assessment functionality. Generates test execution reports and fails pipeline if any test cases fail.

```
------------------------------------------------------------
  >>> STAGE 2: TEST <<<
------------------------------------------------------------
  Loading tests from 'tests/' folder...
  Found 29 test(s) to run.

test_average_is_rounded (test_edtech_app.TestCalculateAverage.test_average_is_rounded)
Average should be rounded to 2 decimal places. ... ok
test_average_of_empty_list (test_edtech_app.TestCalculateAverage.test_average_of_empty_list)
Average of empty list should be 0 (no scores yet). ... ok
test_average_of_normal_scores (test_edtech_app.TestCalculateAverage.test_average_of_normal_scores)
Test average of a typical set of scores. ... ok
test_average_of_single_score (test_edtech_app.TestCalculateAverage.test_average_of_single_score)
Average of one score should be that score itself. ... ok
test_average_of_zeros (test_edtech_app.TestCalculateAverage.test_average_of_zeros)
All zeros should give average of 0. ... ok
test_search_finds_matching_courses (test_edtech_app.TestCourseSearch.test_search_finds_matching_courses)
Search for 'python' should return 2 results. ... ok
test_search_is_case_insensitive (test_edtech_app.TestCourseSearch.test_search_is_case_insensitive)
Searching 'PYTHON' and 'python' should give same results. ... ok
test_search_no_results (test_edtech_app.TestCourseSearch.test_search_no_results)
Searching for something not in the catalog should return empty list. ... ok
test_search_partial_keyword (test_edtech_app.TestCourseSearch.test_search_partial_keyword)
Searching 'data' should match courses with 'data' in the name. ... ok
test_grade_A (test_edtech_app.TestGrading.test_grade_A)
Scores 90 and above should give grade A. ... ok
test_grade_B (test_edtech_app.TestGrading.test_grade_B)
Scores 80-89 should give grade B. ... ok
test_grade_C (test_edtech_app.TestGrading.test_grade_C)
Scores 70-79 should give grade C. ... ok
test_grade_D (test_edtech_app.TestGrading.test_grade_D)
Scores 60-69 should give grade D. ... ok
test_grade_F (test_edtech_app.TestGrading.test_grade_F)
Scores below 60 should give grade F (fail). ... ok
test_add_score_of_hundred (test_edtech_app.TestQuizScores.test_add_score_of_hundred)
Score of 100 is valid (perfect score). ... ok
test_add_score_of_zero (test_edtech_app.TestQuizScores.test_add_score_of_zero)
Score of 0 is valid (student got everything wrong). ... ok
test_add_valid_score (test_edtech_app.TestQuizScores.test_add_valid_score)
Test that a valid score (0-100) can be added. ... ok
test_reject_negative_score (test_edtech_app.TestQuizScores.test_reject_negative_score)
Negative scores are invalid and should be rejected. ...   [ERROR] Invalid score: -5. Score must be between 0 and 100.
ok
test_reject_score_above_100 (test_edtech_app.TestQuizScores.test_reject_score_above_100)
Score above 100 is invalid and should be rejected. ...   [ERROR] Invalid score: 105. Score must be between 0 and 100.
```

Stage 3 - AI Analysis Stage: Performs static code analysis using Python AST module to parse source code into abstract syntax trees. Implements rule-based checks for code quality metrics including function length, docstring presence, variable naming conventions, line length compliance, and TODO marker detection. Generates detailed analysis reports with actionable recommendations.

```
>>> STAGE 3: AI CODE ANALYSIS <<<
------------------------------------------------------------
Analyzing: sample_code/edtech_app.py

--- Long Functions ---
  [!] Function 'add_quiz_score' is 26 lines long (recommended max: 15)
  [!] Function 'calculate_average_score' is 21 lines long (recommended max: 15)
  [!] Function 'get_grade_from_score' is 28 lines long (recommended max: 15)
  [!] Function 'generate_student_report' is 37 lines long (recommended max: 15)
  [!] Function 'search_courses' is 21 lines long (recommended max: 15)

--- Missing Docstrings ---
  [!] Function 'process_batch_enrollment' is missing a docstring

--- Short Variable Names ---
  [!] Variable 's' at line 229 has a very short name

--- TODO Comments ---
  [!] Line 223 has a TODO/FIXME comment

--- Long Lines (Style) ---
  [!] Line 44 is 82 chars long (max: 79)
  [!] Line 45 is 87 chars long (max: 79)
  [!] Line 83 is 84 chars long (max: 79)
  [!] Line 169 is 85 chars long (max: 79)
  [!] Line 228 is 88 chars long (max: 79)

--- Test Suggestions ---
  [!] Write a test called 'test_add_quiz_score()'
  [!] Write a test called 'test_calculate_average_score()'
  [!] Write a test called 'test_get_grade_from_score()'
  [!] Write a test called 'test_generate_student_report()'
  [!] Write a test called 'test_search_courses()'
  [!] Write a test called 'test_process_batch_enrollment()'
  [!] Write a test called 'test_enroll_in_course()'
  [!] Write a test called 'test_get_enrolled_courses()'

Total issues/suggestions: 21

AI Analysis complete. See findings above.
```

Stage 4 - Deploy Stage: Simulates deployment process by creating deployment artifacts and logging deployment metadata. In production scenarios, this stage would integrate with cloud platforms or container orchestration systems for actual deployment execution.

```
------------------------------------------------------------
  >>> STAGE 4: DEPLOY (Simulated) <<<
------------------------------------------------------------
  Target Environment : STAGING
  Build Number       : BUILD-20260416-4090
  Application        : EdTech Learning Platform v1.0.0

  [DEPLOY] Connecting to deployment server...
  [DEPLOY] Packaging application files...
  [DEPLOY] Uploading code to server...
  [DEPLOY] Stopping old version of application...
  [DEPLOY] Installing new version...
  [DEPLOY] Running database migrations...
  [DEPLOY] Starting new version of application...
  [DEPLOY] Running health check...
  [DEPLOY] Deployment verification complete!

  [INFO] Deployment log saved to: reports/deployment_log.txt

  [ PASSED ] DEPLOY completed successfully!

============================================================
   PIPELINE COMPLETED SUCCESSFULLY!
============================================================
   Build Number : BUILD-20260416-4090
   Duration     : 12 second(s)
   Finished     : 2026-04-16 21:07:52
   Reports      : reports/ folder
============================================================
```

2.3 Information Flow

The pipeline orchestrator initiates execution and maintains pipeline state throughout the workflow. Each stage receives control from the orchestrator, performs its designated operations, and returns execution status. Build stage output includes environment validation results. Test stage produces test execution summaries and failure details if applicable. AI analysis stage generates comprehensive code quality reports stored in the reports directory. Deploy stage creates deployment logs and artifact manifests. All stages output console logs for real-time monitoring and debugging. The orchestrator aggregates stage results to determine overall pipeline success or failure status.

2.4 Component Design

The system is composed of multiple modular components, each responsible for a specific function in the CI/CD pipeline.

- **Pipeline Orchestrator Component:**
Controls the overall workflow execution. It manages stage sequencing, handles errors, and ensures smooth execution of the pipeline.

- **Build Stage Component:**
Validates the Python environment, checks required files, and ensures dependencies are available.

- **Test Stage Component:**
Uses the unittest framework to discover and execute test cases. It collects results and generates test reports.

- **AI Analysis Engine Component:**
Performs static code analysis using Abstract Syntax Tree (AST). It checks code quality such as function length, naming conventions, docstrings, and line length.

- **Deploy Stage Component:**
Simulates deployment by creating artifacts and logging deployment information.

- **EdTech Application Component:**
Includes modules for student management, course handling, and assessments.

- **Reporting Component:**
Generates logs and reports for analysis and debugging.

2.5 Key Design Consideration

The system design is based on the following important principles:

- **Modularity:**
Each stage is independent and can be modified without affecting others.
- **Fail-Fast Principle:**
The pipeline stops immediately if any stage fails, saving time and resources.
- **Zero External Dependencies:**
Only Python standard libraries are used to ensure compatibility.
- **Extensibility:**
New stages or rules can be easily added.
- **Observability:**
Logs and reports provide full visibility of pipeline execution.
- **Platform Independence:**
Works on both local systems and cloud platforms like GitHub Actions.

2.6 API Catalogue

The system exposes several APIs for different pipeline stages:

- **Build Stage API:**
run_build_checks() → Returns (success, message)
- **Test Stage API:**
run_tests() → Returns (success, message)
- **AI Analysis API:**
run_ai_analysis() → Returns (success, message) analyze_code(filepath) → Returns issues
generate_report(results) → Generates report
- **Deploy Stage API:**
run_deployment() → Returns (success, message) create_deployment_log() → Creates logs
- **Application APIs:**
Student: add_course(), get_courses()
Course: enroll_student(), get_enrolled_students()
Assessment: submit_score(), get_scores()

3. Data Design

3.1 Data Model

Pipeline Execution State contains stage_name (string), status (success/failure/running), start_time (timestamp), end_time (timestamp), error_message (string optional). Analysis Report Model contains file_path (string), issues_found (list), issue_type (string), line_number (integer), severity (low/medium/high), recommendation (string). Test Results Model contains test_name (string), test_status (pass/fail/skip), execution_time (float), failure_reason (string optional). Deployment Artifact Model contains artifact_name (string), version (string), build_timestamp (timestamp), deployment_status (string), target_environment (string).

3.2 Data Access Mechanism

File System Operations: Analysis reports and deployment logs stored as text files in reports directory. Python built-in file I/O operations handle read/write operations. In-Memory
Processing: Pipeline state maintained in Python objects during execution. Test results collected in memory before report generation. Configuration Management: Pipeline configuration through Python variables and environment variables. GitHub Actions workflow defined in YAML configuration file. No Database: Current implementation does not require persistent database storage. All data is transient or file-based for simplicity and portability.

3.3 Data Retention Policies

Analysis reports and deployment logs are retained indefinitely in the reports directory for historical reference and trend analysis. In production deployments, implement log rotation policies to manage disk space. Recommended retention period is 90 days for detailed logs with archival of summary metrics for longer term analysis. GitHub Actions execution logs are retained according to GitHub platform policies.

3.4 Data Migration

Not applicable for current implementation as system does not maintain persistent state requiring migration. Future enhancements adding database storage would require migration strategies for schema updates and data transformations.

4. Interfaces

Command Line Interface: Primary interface for local pipeline execution through run_pipeline.py. Accepts standard input/output for user interaction and status reporting. GitHub Actions Interface: Workflow defined in .github/workflows/pipeline.yml configures automated execution on push events. Integrates with GitHub event system for trigger management. File System Interface: Reads source code files from sample_code and tests directories. Writes output reports to reports directory. Module Interface: Each pipeline stage exposes standardized function signatures for orchestrator integration. Python unittest interface for test discovery and execution.

5. State and Session Management

Pipeline execution is stateless with each run being independent. No session management required as system does not maintain user sessions or long-running processes. Each pipeline execution creates new runtime context that is discarded upon completion. State information during execution maintained in memory only.

6. Caching

Current implementation does not utilize caching mechanisms due to lightweight nature of operations. Python bytecode caching is leveraged automatically for imported modules. Future enhancements could implement caching for analysis results of unchanged files to improve performance in large codebases.

7. Non-Functional Requirements

7.1 Security Aspects

Code Execution Isolation: Pipeline stages execute in controlled environment with no external network access requirements. Input Validation: File paths validated before processing to prevent directory traversal attacks. Access Control: GitHub Actions workflow requires repository write permissions limiting who can trigger pipeline execution. Secure Credential Management: No hardcoded credentials in source code. Environment variables used for sensitive configuration. Audit Logging: All pipeline executions logged with timestamps for security auditing and compliance.

7.2 Performance Aspects

Execution Time: Complete pipeline execution completes within 30 seconds for typical EdTech application codebase. Scalability: Architecture supports parallel execution of independent stages in future enhancements. Resource Utilization: Minimal memory footprint using streaming processing where applicable. CPU intensive operations limited to AST parsing during analysis stage. Reliability: Fail-fast approach ensures predictable execution time. No timeout issues due to lightweight operations.

8. References

Python Official Documentation - https://docs.python.org/3/. Python unittest Framework Documentation. Python AST Module Documentation. GitHub Actions Documentation - https://docs.github.com/actions. PEP 8 Style Guide for Python Code. Continuous Integration and Continuous Deployment Best Practices. Martin Fowler - Continuous Integration. The DevOps Handbook by Gene Kim et al.